

E.3 Runtime versus number of nodes in simulation graph.

We measure the time to execute one step of simulation as a function of the input graph on different scenes in Movi-C dataset. For both SDF-Sim and FIGNet*, the runtime grows linearly with the number of the nodes in the graph. The runtime of SDF-Sim is consistently lower than FIGNet*. On large meshes with ~ 8000 nodes and ~ 18000 peak edges, SDF-Sim has a 52% smaller runtime compared to the FIGNet* baseline.

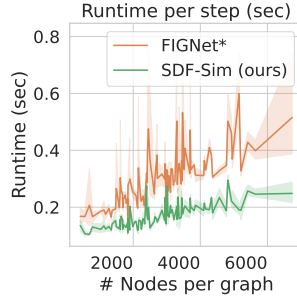


Figure S2: **Runtime per simulation step** w.r.t. the total number of nodes in the scene. The runtime evaluations were performed on the Movi-C test set on Nvidia A100 GPU. FIGNet baseline runs OOM on Movi-C and is not shown here.

E.4 Memory of the SDF versus a mesh

A key advantage of learned SDFs is that they provide a compact representation of complex object shapes compared to meshes. In Figure S3, we compare the sizes of meshes from Movi-C dataset (red) to the sizes of the SDF model parameters (purple) in MiB, assuming both are stored as pickle files on disk. While the size of the meshes grows with the number of mesh nodes, memory requirements of the SDFs do not depend on the complexity of the object, as both simple and intricate shapes are modeled with the same size MLP. We see that our MLP with 8 layers of 128 units has similar memory demands to meshes with 15,000 nodes, which is 4x less memory than the largest meshes in Movi-C. Thus, SDF-Sim can potentially use less memory to store the object shapes: while FIGNet and FIGNet* require to store the entire object mesh, SDF-Sim needs only a compact SDF and a set surface nodes, which, as we will show in section E.5, can be relatively small.

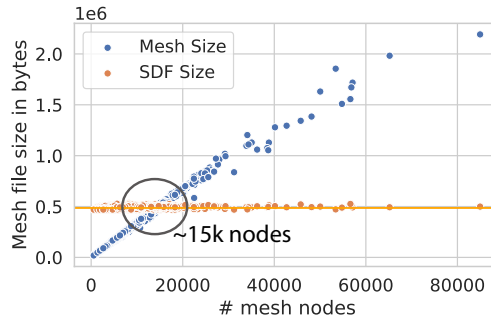


Figure S3: **Memory footprint of storing the mesh versus storing the SDF weights** w.r.t. the number of mesh nodes, evaluated on 930 objects from Movi-C.

E.5 Re-sampling object surface nodes using an SDF

Simulation accuracy heavily depends on the quality of the object shapes, typically a mesh. However, not all meshes are suitable for simulation. Object meshes, either created by artists or obtained by scanning a real-world object, often consist of thousands of nodes, and the nodes are concentrated around on the fine details of the shape (an example shown in Figure S4(a)). To be able to simulate the objects, the shapes are often approximated by hand-crafted convex *collision meshes* (Figure S4(b)). But such meshes might not always exist, and might not be optimal for GNN-based simulators like FIGNet. However, mesh-based simulators like FIGNet, typically use these collision meshes, as changing a mesh while retaining a connected surface is not easy.

In contrast, SDF-Sim does not require a connected mesh; it relies only on surface nodes and the learned SDF to perform a simulation step. This gives us full flexibility to choose the number and the location of graph nodes. Varying the number of surface nodes gives us a tighter handle controlling the tradeoff between accuracy, simulation speed and memory.

To create a new set of surface nodes, we can *sample* the surface nodes from a learned SDF. We initialize $K = N^3$ equally-spaced points in the volume of a cube which encompasses the object. We project these points to the object surface using Eq. 1, with minor filtering (more details on this sampling procedure described below). With this strategy, we get points that evenly cover the entire object surface (Figure S4(c)). Finally, we take the SDF-Sim simulator trained on Movi-C and replace each object’s mesh nodes with the ones sampled from an SDF *at test time*. Figure 6 shows that with this sampling strategy, we can maintain the simulation accuracy using 73.5% fewer nodes (hence smaller memory) compared to SDF-Sim with Movi-C collision meshes, used elsewhere in the paper. Specifically, in Figure S4(d) the models with $K = 7^3$ and $K = 8^3$ (dark brown bars) have similar translation error (0.23) compared to the model with original node distribution (green bar, 0.24). However, in Figure S4(e) the model $K = 7^3$ requires only 934.19 nodes per scene on average (fourth orange column), while the model with the original Movi-C nodes needs 3530.13 nodes (green column) to perform the same simulation.

Here we perform this experiment at test time only, but it would be straightforward to also *train* the SDF-Sim with the nodes sampled from an SDF, and experiment with sampling strategies for optimal prediction performance. This creates the opportunity to use a learned SDF as the sole representation of shape without ever requiring a mesh.

Details on sampling the surface nodes from an SDF We start with a uniform 3D grid of $N \times N \times N$ points, located within an interval $[-0.7, 0.7]$, to ensure that the grid is larger than the object (recall that the object meshes were rescaled to $[-0.5, 0.5]$ before training an SDF). Placing the query points on a grid ensures that the corresponding projected points would cover the entire object surface, including concave parts like inner side of a shoe or a knot. Then, we project these points on the surface: $\mathbf{y}^* = \mathbf{y} - f_\theta(\mathbf{y})\nabla f_\theta(\mathbf{y})$. We generally find that the trained SDF functions are fairly accurate and the projected points are close to the surface. Nevertheless, we additionally filter the points $f_\theta(\mathbf{y}^*) < 0.01$ to ensure that $\mathbf{y}^*$ are exactly on the surface as it otherwise would impact the accuracy of the simulation. Finally, some areas of the shape might have clusters of points. We additionally downsample projected points using `voxel_down_sample` function from open3D library [42], such that at most one point is present per voxel of size 0.025 to further reduce the number of points.

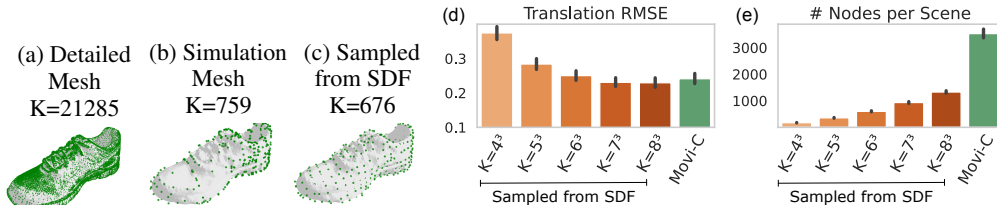


Figure S4: Shoe object from Movi-C, with vertices shown in green: (a) the original high-resolution mesh for rendering, (b) collision mesh for simulation, (c) nodes sampled on a grid and projected to the learned SDF). We can achieve favorable tradeoffs between error (d) and node count (e) by varying the dimensions K of the sample grid (orange bars). The green bar represents SDF-Sim that uses the nodes from the original Movi-C collision mesh, as used elsewhere in the paper.

E.6 Quantifying the source of SDF-Sim error

A natural question to ask is, where does the increased error (in comparison to FIGNet*) of SDF-Sim on Movi B/C comes from. A possible way to quantify this would be to train an SDF-Sim architecture using the accurate distances directly computed from a mesh. We note that correctly estimating *signed* distances is actually hard and is subject to edge cases. We ran an experiment where we train SDF-Sim on Movi-C with estimates of the *unsigned* distance (UDF) computed from the mesh via BVH. We achieve similar Translation RMSE on Movi-C (0.23 with mesh-based UDFs versus 0.24 with learned SDF). While it is not a one-to-one comparison, this result does indicate that our method is relatively robust to SDF accuracy, and the learned simulator can learn to correct for minor inaccuracies.